

Laporan Praktikum Minggu Ke-5 Kontrol Cerdas

Minggu ke-5

Nama : Ika Putri Ayuningtia

NIM : 224308057

Kelas : TKA 7C

Akun Github (Tautan) : <https://github.com/ikaputriayuningtia-sketch>

1. Judul Percobaan

Deep Reinforcement Learning untuk Kontrol Kompleks

2. Tujuan Percobaan

Tujuan dari percobaan pada praktikum kali ini, sebagai berikut:

1. Memahami konsep dasar kontrol cerdas (intelligent control systems).
2. Memahami konsep Deep Reinforcement Learning (DRL) dalam kontrol sistem kompleks.
3. Mengimplementasikan Deep Q-Network (DQN) untuk kontrol otomatis.
4. Menganalisis performa DRL dibandingkan dengan metode kontrol konvensional.

3. Landasan Teori

Deep Reinforcement Learning (DRL) merupakan pengembangan dari *Reinforcement Learning (RL)* yang menggunakan jaringan saraf (*Neural Network*) untuk menangani sistem dengan kondisi yang kompleks dan kontinu. Dalam RL, agen belajar melalui interaksi dengan lingkungan untuk memperoleh *reward* maksimum, sedangkan pada DRL proses pembelajaran dilakukan dengan memanfaatkan kemampuan *Deep Learning* dalam mengenali pola dan memprediksi nilai tindakan (*Q-value*) secara lebih efisien (Sutton & Barto, 2018).

Salah satu algoritma utama dalam DRL adalah Deep Q-Network (DQN). DQN menggantikan tabel Q klasik dengan jaringan saraf yang berfungsi memperkirakan nilai Q dari setiap aksi berdasarkan kondisi lingkungan. Algoritma ini menggunakan konsep *experience replay* untuk meningkatkan stabilitas pelatihan

dan strategi *epsilon-greedy* untuk menyeimbangkan eksplorasi dan eksploitasi (Mnih et al., 2015).

DQN banyak digunakan dalam sistem kontrol kompleks seperti robot otonom, kendaraan tanpa pengemudi, dan optimasi energi karena mampu beradaptasi terhadap perubahan lingkungan tanpa memerlukan model matematis sistem yang eksplisit.

4. Diskusi

1. Kelebihan dan Kelemahan DQN dibandingkan Metode Q-Learning Klasik

Deep Q-Network (DQN) memiliki sejumlah kelebihan dibandingkan metode Q-Learning klasik, terutama dalam hal kemampuan menangani sistem yang kompleks dan ruang keadaan yang besar. Pada Q-Learning konvensional, proses pembelajaran dilakukan menggunakan tabel Q (*Q-Table*), yang berisi nilai untuk setiap pasangan keadaan dan aksi. Pendekatan ini efektif untuk sistem sederhana dengan jumlah keadaan terbatas, tetapi menjadi tidak efisien ketika ruang keadaannya sangat besar atau bersifat kontinu. DQN mengatasi keterbatasan ini dengan menggunakan jaringan saraf tiruan (*Neural Network*) untuk memperkirakan nilai Q secara langsung, sehingga mampu bekerja pada lingkungan yang lebih kompleks dan dinamis.

Selain itu, DQN juga memiliki mekanisme *experience replay* yang memungkinkan pengambilan sampel pengalaman secara acak selama proses pelatihan. Teknik ini membantu mengurangi korelasi antar data dan meningkatkan stabilitas pembelajaran. Strategi *epsilon-greedy* yang diterapkan dalam DQN juga membantu menjaga keseimbangan antara eksplorasi dan eksploitasi, sehingga agen dapat belajar lebih efektif dari lingkungannya.

Namun, di sisi lain, DQN juga memiliki beberapa kelemahan. Algoritma ini memerlukan sumber daya komputasi yang tinggi karena melibatkan proses pelatihan jaringan saraf yang kompleks. Waktu pelatihannya pun relatif lebih lama dibandingkan Q-Learning klasik. Selain itu, performa DQN sangat bergantung pada pemilihan parameter seperti *learning rate*, *discount factor*, dan ukuran jaringan. Jika parameter tidak diatur dengan tepat, model dapat mengalami *overfitting* atau ketidakstabilan dalam pembelajaran. Dengan demikian, meskipun DQN

menawarkan kemampuan yang lebih canggih, implementasinya memerlukan keahlian teknis dan perancangan yang hati-hati.

2. Adaptasi DQN untuk Kontrol Sistem Dinamis seperti Drone atau Robot Industri

Untuk mengadaptasi DQN dalam kontrol sistem dinamis seperti drone atau robot industri, diperlukan beberapa penyesuaian agar algoritma dapat bekerja secara efektif dalam lingkungan yang terus berubah. Langkah pertama adalah mendefinisikan *state* dan *action* dengan tepat. Pada sistem seperti drone, *state* dapat mencakup posisi, orientasi, dan kecepatan, sementara *action* dapat berupa perubahan sudut baling-baling atau gaya dorong. Data dari sensor seperti IMU, kamera, atau encoder digunakan sebagai input agar agen DQN memperoleh informasi yang akurat mengenai kondisi sistem secara real time.

Pelatihan DQN untuk sistem dinamis biasanya dimulai di lingkungan simulasi seperti OpenAI Gym, Gazebo, atau PyBullet. Simulasi digunakan untuk menghindari risiko kerusakan perangkat nyata selama proses pembelajaran awal. Setelah model menunjukkan performa yang stabil di simulasi, hasilnya dapat diterapkan pada perangkat fisik melalui proses *transfer learning*. Selain itu, dalam penerapan dunia nyata, DQN sering dikombinasikan dengan metode kontrol konvensional seperti PID atau Fuzzy Logic untuk menjaga kestabilan sistem saat model masih dalam tahap adaptasi.

Agar sistem dapat belajar dengan baik, *reward function* harus dirancang dengan cermat sesuai tujuan kontrol, misalnya menjaga keseimbangan drone atau mengarahkan robot menuju posisi target dengan efisien. Melalui kombinasi desain yang tepat, DQN mampu beradaptasi secara mandiri terhadap perubahan lingkungan dan menghasilkan kontrol yang presisi pada sistem dinamis yang kompleks.

5. Assignment

Dalam tugas minggu kelima ini dikembangkan sistem pengenalan gestur tangan real-time menggunakan Support Vector Machine (SVM) dan MediaPipe yang terintegrasi dengan OpenCV. Proses dimulai dengan memuat dataset keypoint tangan menggunakan `pd.read_csv()`, kemudian menyesuaikan nama kolom agar seragam. Dataset diasumsikan memiliki label pada kolom pertama dan noise pada

kolom terakhir, sehingga fitur (X) diambil dari kolom ke-2 hingga sebelum kolom terakhir, sedangkan label (y) berasal dari kolom pertama. Data dibagi menjadi 80% untuk pelatihan dan 20% untuk pengujian menggunakan `train_test_split()`.

Model SVM dengan kernel linear dilatih menggunakan `SVC(kernel='linear')` dan diuji pada data uji untuk menghitung tingkat akurasi melalui `accuracy_score()`. Setelah pelatihan, model diterapkan dalam deteksi gestur tangan secara real-time dengan MediaPipe dan OpenCV. Kamera menangkap citra, mengonversinya ke format RGB, lalu mendeteksi landmark tangan menggunakan `hands.process()`. Jika tangan terdeteksi, koordinat x, y, dan z setiap titik diekstraksi dan dimasukkan ke model SVM untuk memprediksi gestur. Hasil prediksi ditampilkan di layar menggunakan `cv2.putText()`.

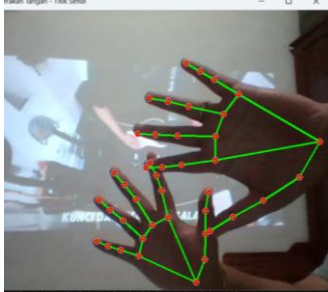
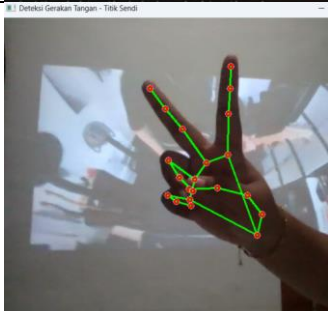
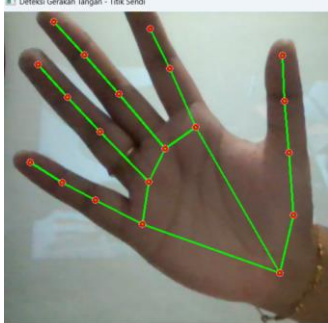
Dengan menggabungkan MediaPipe untuk pelacakan titik kunci tangan dan SVM untuk klasifikasi gestur, sistem ini mampu mengenali posisi serta gerakan tangan secara akurat dalam waktu nyata, menunjukkan penerapan efektif antara *machine learning* dan *computer vision* dalam analisis gerakan manusia. Berikut ini adalah fungsi dari kode program yang digunakan seperti pada tabel berikut ini :

No	Bagian Kode / Fungsi	Penjelasan
1	<code>import cv2</code>	Mengimpor pustaka OpenCV untuk menangani pengolahan citra dan akses kamera.
2	<code>import mediapipe as mp</code>	Mengimpor pustaka MediaPipe untuk melakukan deteksi dan pelacakan tangan secara real-time.
3	<code>mp_hands = mp.solutions.hands</code>	Menginisialisasi modul Hands dari MediaPipe yang berfungsi mendeteksi dan melacak titik tangan (landmarks).
4	<code>mp_drawing = mp.solutions.drawing_utils</code>	Menginisialisasi fungsi utilitas untuk menggambar hasil deteksi landmark di atas frame video.
5	<code>cap = cv2.VideoCapture(0)</code>	Membuka koneksi ke kamera (0 berarti kamera bawaan laptop). Jika ada kamera eksternal, indeks bisa diganti (misal 1 atau 2).
6	<code>with mp_hands.Hands(...) as hands:</code>	Membuat objek deteksi tangan dengan parameter: - <code>max_num_hands=2</code> : mendeteksi maksimal dua tangan. - <code>min_detection_confidence=0.7</code> : batas minimal kepercayaan

		deteksi. - min_tracking_confidence=0.7: batas minimal kepercayaan pelacakan.
7	<code>ret, frame = cap.read()</code>	Membaca satu frame dari kamera. <code>ret</code> bernilai <i>True</i> jika pembacaan berhasil.
8	<code>cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)</code>	Mengubah format warna dari BGR (OpenCV) ke RGB (MediaPipe membutuhkan format RGB).
9	<code>results = hands.process(rgb_frame)</code>	Memproses frame RGB untuk mendeteksi tangan dan menghasilkan koordinat titik-titik (landmarks).
10	<code>if results.multi_hand_landmarks:</code>	Mengecek apakah ada tangan yang terdeteksi pada frame.
11	<code>for hand_landmarks in results.multi_hand_landmarks:</code>	Melakukan iterasi untuk setiap tangan yang terdeteksi (bisa satu atau dua).
12	<code>mp_drawing.draw_landmarks(...)</code>	Menggambar titik-titik sendi dan koneksi antar sendi tangan di atas frame video dengan warna dan ketebalan tertentu.
13	<code>cv2.imshow("Deteksi Gerakan Tangan - Titik Sendi", frame)</code>	Menampilkan hasil deteksi tangan secara real-time di jendela OpenCV.
14	<code>if cv2.waitKey(1) & 0xFF == ord('q'):</code>	Memberi opsi keluar dari program jika tombol 'q' ditekan di keyboard.
15	<code>cap.release()</code>	Menutup koneksi kamera setelah program berhenti.
16	<code>cv2.destroyAllWindows()</code>	Menutup semua jendela tampilan OpenCV.

6. Data dan Output Hasil Pengamatan

Tabel berikut menunjukkan hasil pengamatan deteksi gerakan tangan menggunakan sistem berbasis MediaPipe. Setiap gerakan dianalisis berdasarkan jumlah tangan yang terdeteksi, jumlah titik sendi per tangan, posisi atau titik penting pada tangan, serta keterangan visualisasi output yang dihasilkan.

No	Dokumentasi	Deteksi	Jumlah Titik Sendi per Tangan	Posisi/Titik Penting	Keterangan Output Visual
1		Gerakan Tangan terbuka lebar Jumlah Tangan Terdeteksi 2	21 titik per tangan	Semua jari terbuka, jari-jari terlihat jelas	Garis hijau menghubungkan sendi, titik merah di setiap sendi
2		Gerakan Tangan membentuk "peace" Jumlah Tangan Terdeteksi 1	21 titik	Jari telunjuk & jari tengah terangkat, jari lain menekuk	Garis hijau mengikuti bentuk jari, visualisasi sendi tetap akurat
3		Gerakan Tangan terbuka normal Jumlah Tangan Terdeteksi 1	21 titik	Semua jari terbuka, posisi natural	Deteksi sendi stabil, tidak ada tangan kedua

7. Kesimpulan

Berdasarkan praktikum dan analisis yang telah dilakukan, dapat disimpulkan bahwa program yang dikembangkan berhasil melakukan klasifikasi gestur tangan secara real-time menggunakan kombinasi model SVM dan MediaPipe. Tingkat akurasi model sangat bergantung pada kualitas dataset serta parameter yang digunakan, sehingga optimasi melalui perbaikan dataset, penyetelan model, dan peningkatan performa real-time dapat meningkatkan hasil deteksi. Sistem ini mampu mengenali gerakan tangan dengan cukup baik, meskipun masih menghadapi tantangan terkait noise dan kondisi pencahayaan. Perlu dicatat bahwa sistem hanya mendeteksi keberadaan tangan dan maksimal dua tangan

sekaligus, sehingga tidak dapat mengenali objek lain atau lebih dari dua tangan dalam satu waktu.

8. Saran

Berdasarkan hasil praktikum dan analisis, beberapa saran perbaikan berikut dapat dilakukan untuk meningkatkan akurasi dan performa sistem.

1. **Peningkatan Dataset:** Menambah jumlah dan variasi sampel agar mencakup berbagai kondisi pencahayaan dan posisi tangan, sehingga model lebih general dan akurat.
2. **Optimasi Model dan Performa Real-Time:** Melakukan hyperparameter tuning pada SVM (misal C dan gamma), optimasi kode, kompresi gambar, serta post-processing seperti time smoothing untuk prediksi lebih stabil.
3. **Peningkatan Kapasitas Deteksi:** Memperluas kemampuan sistem agar dapat mendeteksi lebih dari dua tangan atau objek relevan lainnya.

10. Daftar Pustaka

- Mnih, V., et al. (2015). *Human-level control through deep reinforcement learning*. *Nature*, 518(7540), 529–533.
- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.